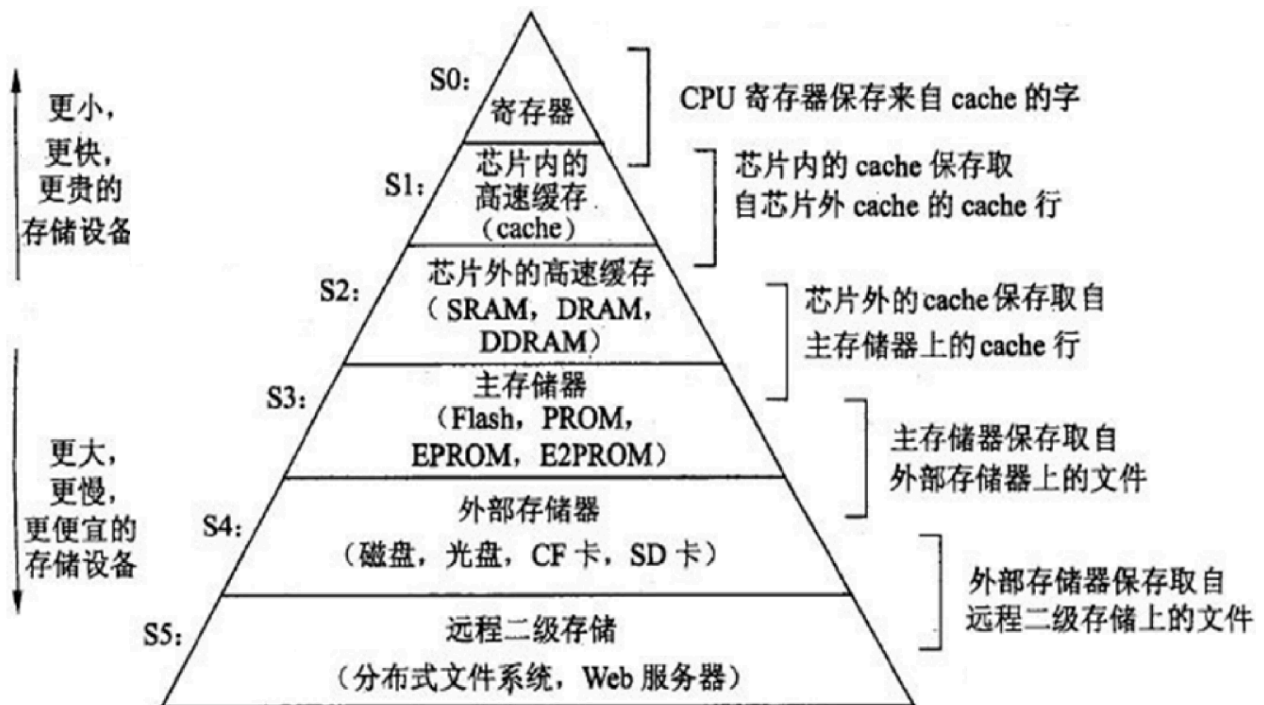


Chapter6 存储器

May 31st, Rafael

存储器层次结构



- 这里有个错误：**DRAM**（动态随机存取存储器）和 **DDRAM**（双倍数据速率 DRAM）实际上是主内存（也就是我们平常说的 **RAM**）的实现技术，所以应该放在**S3主存储器**而不是**S2芯片外的高速缓存**
- 要记住不同器件的速度/容量/成本顺序

存储器的分类

1. 按工作性质/存取方式分类

- **随机存取存储器 Random Access Memory (RAM)**：每个单元读写时间一样，且与各单元所在位置无关。如：内存。现在的DRAM使用行缓存，地址不同可能导致访问时间不一样
- **顺序存取存储器 Sequential Access Memory (SAM)**：数据按顺序从存储载体的始端读出或写入，因而存取时间的长短与信息所在位置有关。例如：磁带。
- **直接存取存储器 Direct Access Memory (DAM)**：直接定位到读写数据块，在读写数据块时按顺序进行。如磁盘。

- **相联存储器 Content Addressed Memory (CAM)或AM**：按内容检索到存储位置进行读写。例如：快表。

2. 按存储介质分类

- **半导体存储器**：双极型，静态MOS型，动态MOS型，RAM，ROM，SSD固态硬盘
- **磁表面存储器**：磁盘（Disk）、磁带（Tape）、硬盘（Hard Disk Drive）
- **光存储器**：CD，CD-RO

3. 按信息的可更改性分类

- **读写存储器 R/W和只读存储器ROM**

4. 按断电后信息的可保存性

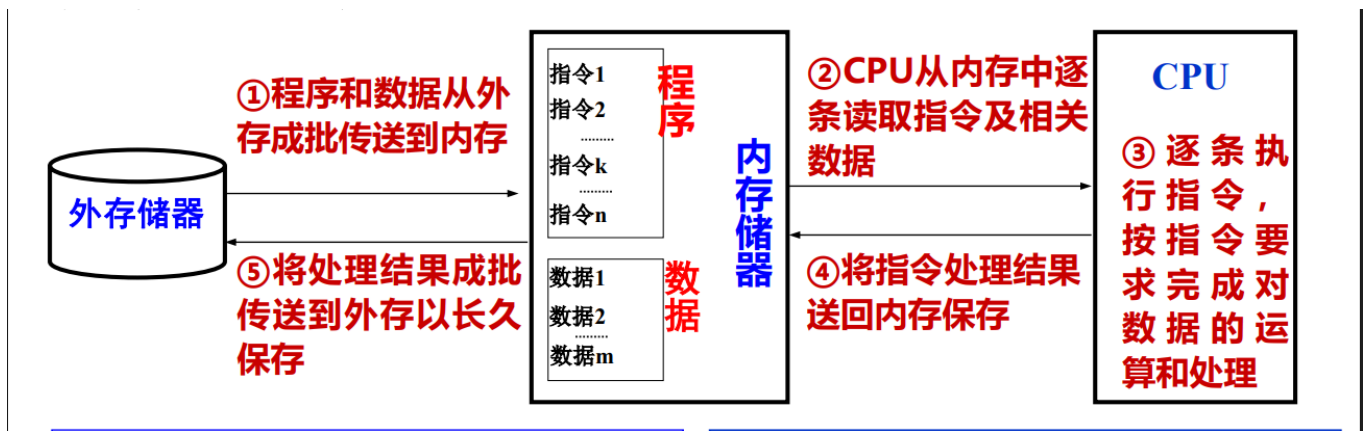
- **非易失（不挥发）性存储器(Nonvolatile Memory)**：信息可一直保留，不需电源维持。如ROM、磁表面存储器、光存储器、闪存等
- **易失（挥发）性存储器(Volatile Memory)**：电源关闭时信息自动丢失。如RAM、Cache等

5. 按功能/容量/速度/所在位置分类

- **寄存器(Register)**：封装在CPU内，用于存放当前正在执行的指令和使用的数据 用触发器实现，速度快，容量小（一个CPU里几~几十个寄存器）
- **高速缓存(Cache)**：位于CPU内部或附近，用来存放当前要执行的局部程序段和数据 用SRAM实现，速度可与CPU匹配，容量小（几MB）
- **内存存储器MM（主存储器Main/Primary Memory）**：位于CPU之外，用来存放已被启动的程序及所用的数据 用DRAM实现，速度较快，容量较大（几GB）。**易失**
- **外存储器AM（辅助存储器Auxiliary / Secondary Storage）**：位于主机之外，用来存放暂不运行的程序、数据或存档文件 用磁表面或光存储器实现，容量比内存大而速度慢，比内存便宜。**非易失**

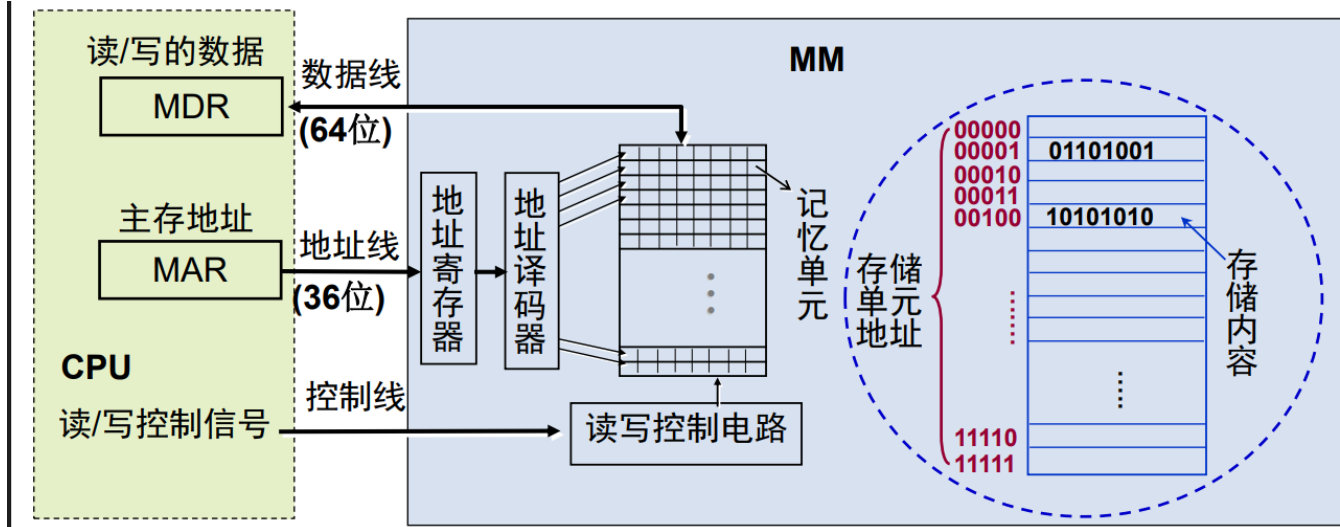
内存和外存

内存、外存和CPU的联系



主存（内存）中有指令和数据，当CPU执行指令的时候会访问主存来取指令、取/存数据

主存的结构



地址译码器的输入是地址，输出地址驱动信号

图中地址线36位，（按字节编址时）可寻址范围是 $0 \sim 2^{36} - 1$ ，主存地址空间64GB

图中数据线64位，（按字节编址时）每次最多访问8字节，给的是首地址（最小地址）

主存地址空间**不等于**主存容量

MDR/MAR: Memory Data/Address Register

主存的性能指标

存储容量：所包含的存储单元的总数（MB/GB）

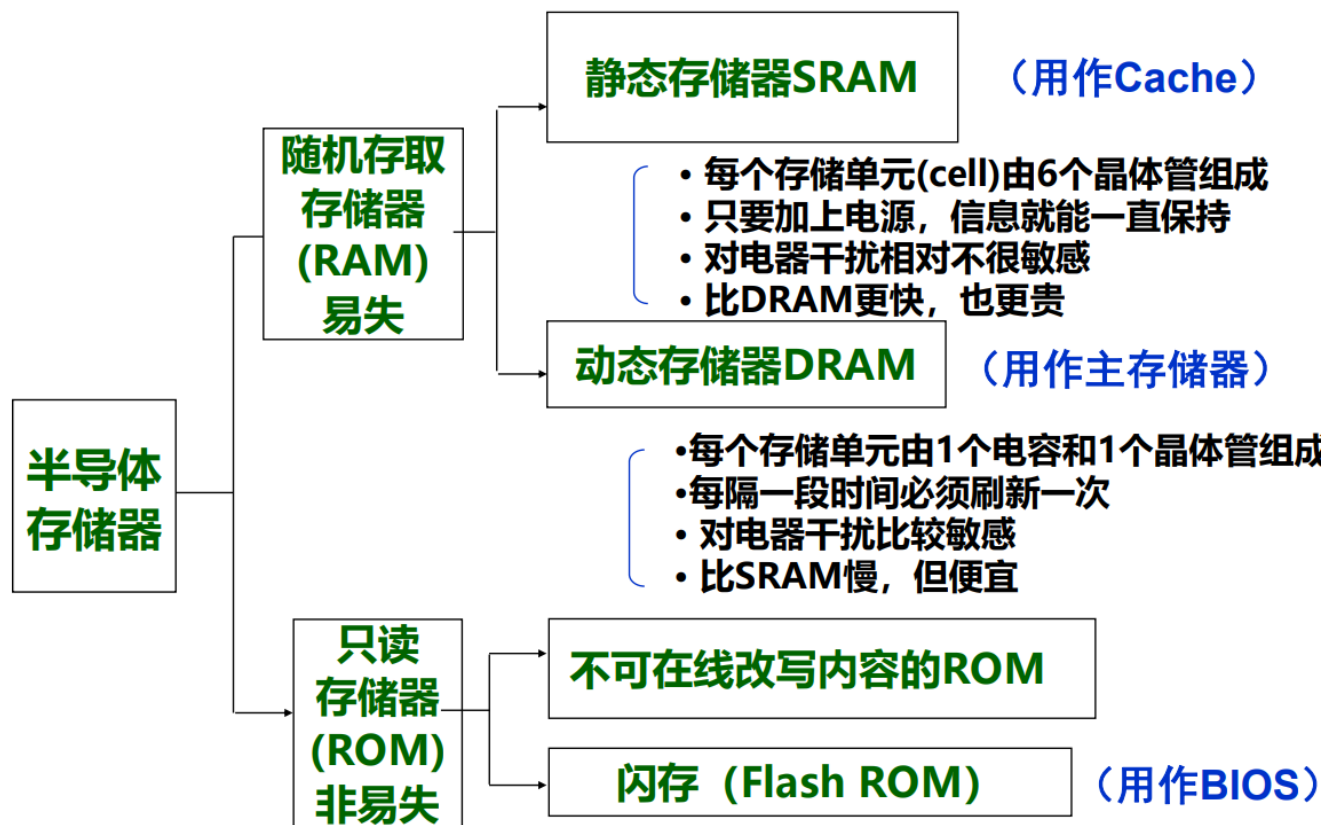
存取时间 T_A ：从CPU送出内存单元的地址码开始，到主存读出数据到CPU所需要的时间（单位：ns, $10^{-9}s$ ）。分读取时间和写入时间

存储周期 T_{MC} ：连读两次访问存储器所需的最小时间间隔，它等于存取时间加上下一次存取开始前所要求的附加时间

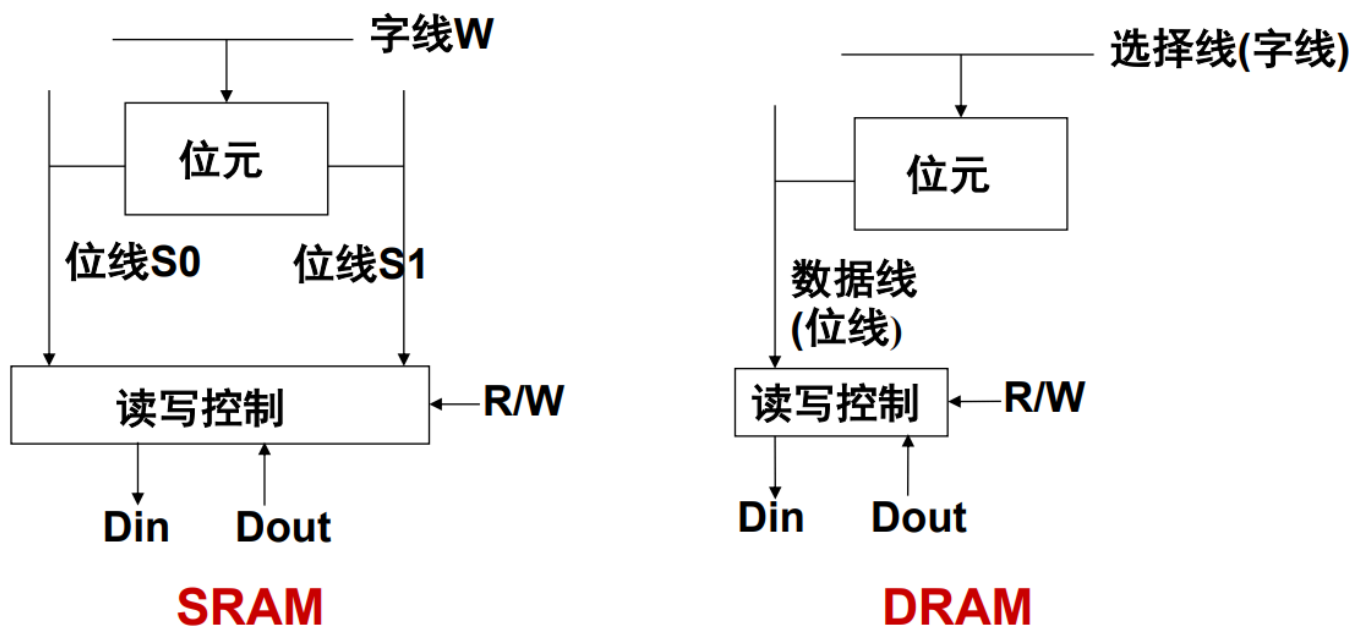
存储器由于读出放大器、驱动电路等都有一段稳定恢复时间，读出后不能立即进行下一次访问。因此， $T_{MC} > T_A$

不同种类半导体存储器的作用

内存由半导体存储器芯片组成



SRAM和DRAM的组织



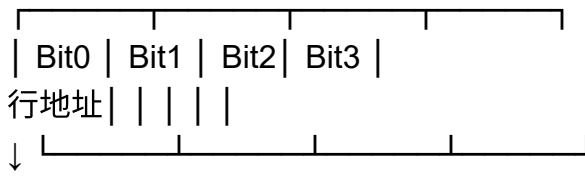
以16M (4M个地址×每个地址4位) 位DRAM为例

$16M\text{位} = 4Mb \times 4 = 2048 \times 2048 \times 4 = 2^{11} \times 2^{11} \times 4 = \text{行} \times \text{列}$, 也就是访问任意位置需要11位行地址和11位列地址 (11根地址线)。但 DRAM 引脚有限, 行地址和列地址不会用两组线传送, 而是“共用”一组地址线, 用先行后列两次传送完成 (分时复用)。

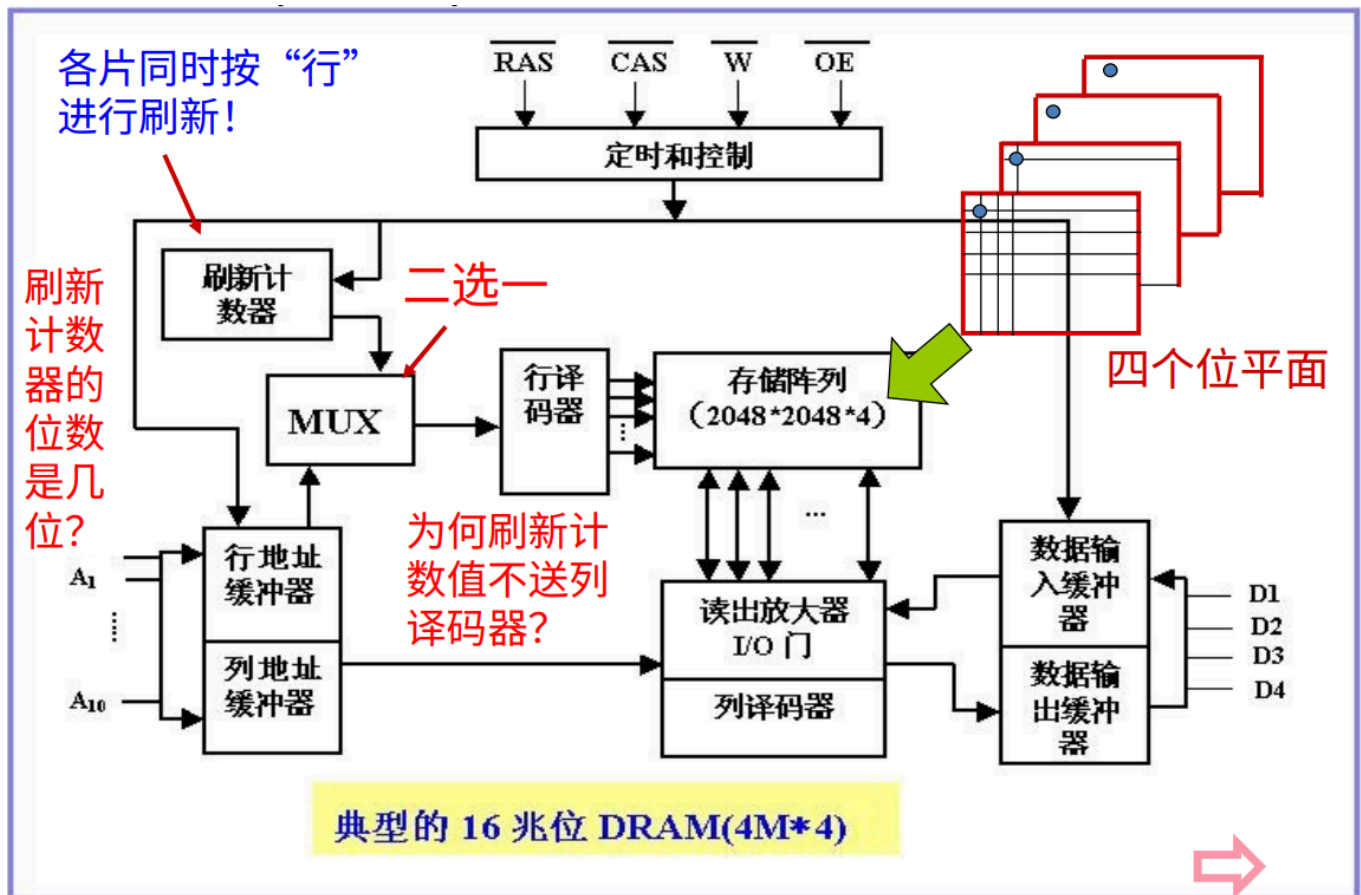
控制先行后列时序的信号叫RAS (行地址通选信号) 和CAS (列地址通选信号)

4位宽意味着每个“行-列”地址对应的是 4 个并位的位元单元, 分别来自4 个位平面 (bit plane), 你可以想成这样

列地址 →



所以需要4个位平面，对相同行、列交叉点的4位一起读/写。先选中一整行，再在这行中选中列



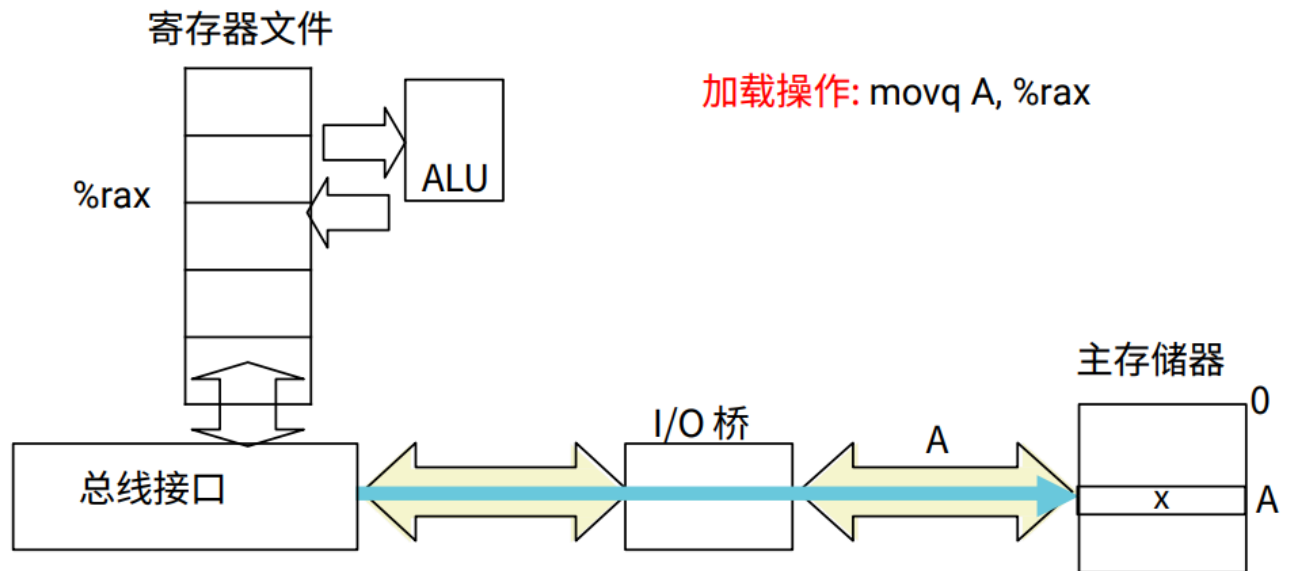
计数器是按行刷新的（所以用不到列译码器），刷新的位数是 $\log_2 2048 = 11$ 位

刷新指的是定期把每一行的数据读出再写回，重新充电

MUX多路选择器根据输入在正常访问CPU送入的地址和自动刷新计数器生成的行地址里选择一个作为行地址输入

存储器读写事务

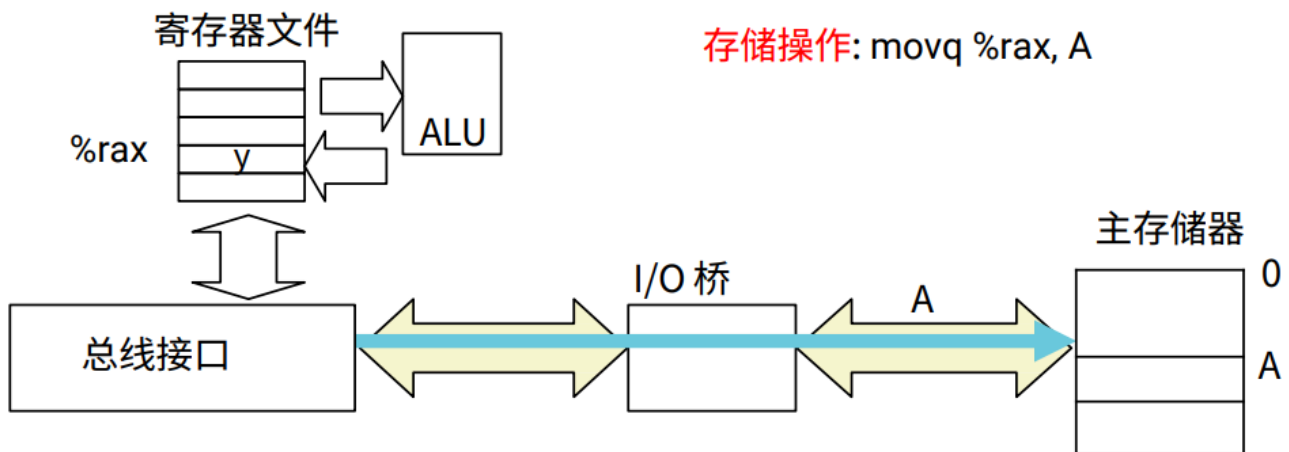
1. 读事务



CPU把地址A放到总线上。主存储器从总线上读地址A，取出字x，然后把x放到总线上。CPU从总线上读入字x，并将其放入寄存器 %rax

`movq A, %rax`：把内存地址A中的8字节内容加载到寄存器 %rax 中

2. 写事务



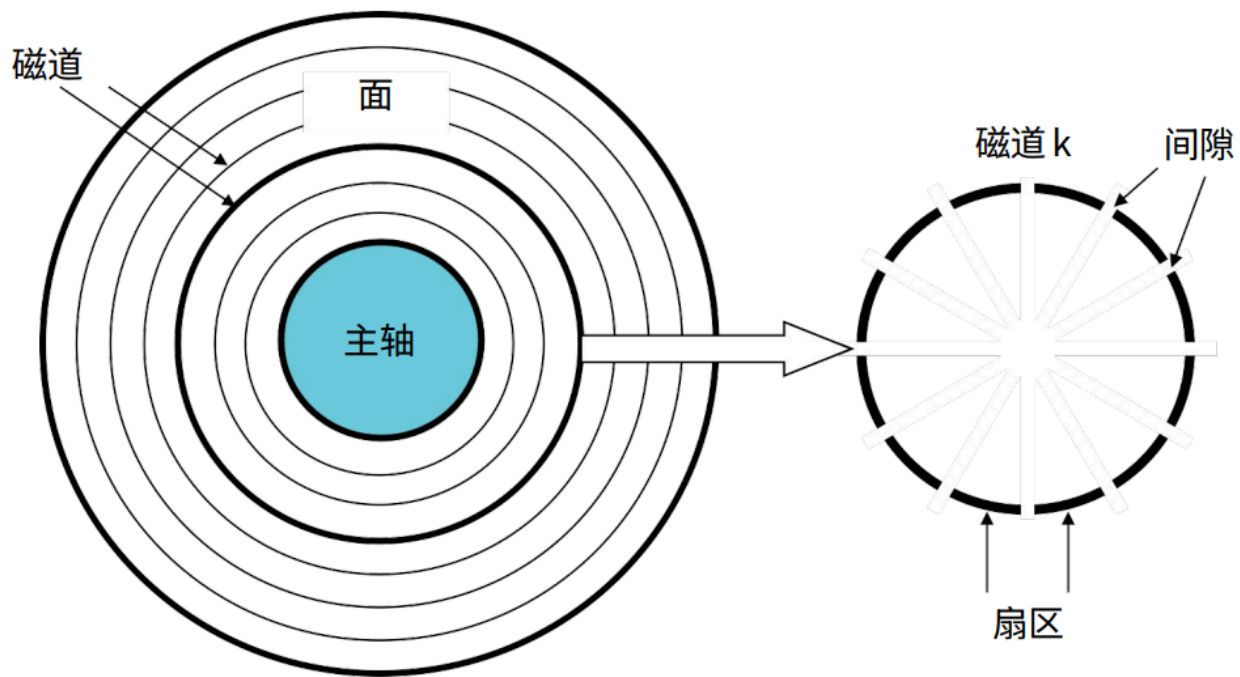
CPU 将地址A放到总线上，主存储器读地址A并等待数据。CPU把字y放到总线上，主存储器从总线上读入字y，并将其存入地址A中。

磁盘

磁盘结构

磁盘由盘片(platter)构成，每个盘片包含两面(surface)。每面由一组称为磁道(track)的同心圆组

成。每个磁道划分为一组扇区(sector)，扇区之间由间隙(gap) 隔开。



磁盘容量

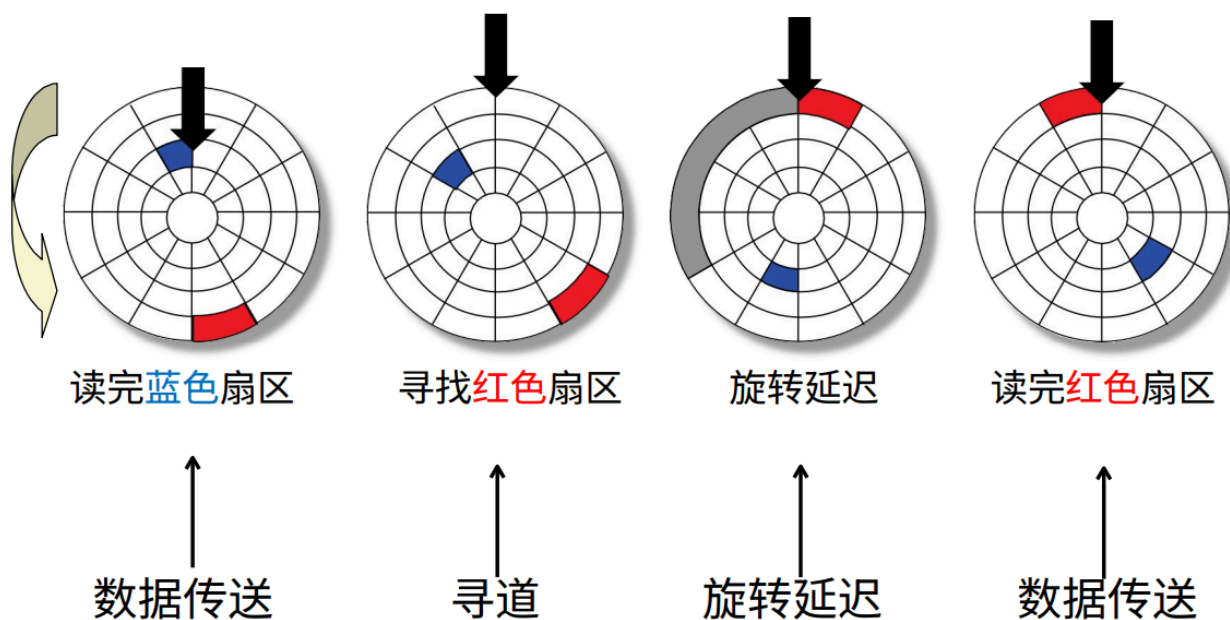
容量Capacity: 磁盘上可以存储的最大位数

计算磁盘容量:

$$\text{容量} = (\text{字节数/扇区}) \times (\text{平均扇区数量/磁道}) \times (\text{磁道数量/面}) \times (\text{面数/盘片}) \times (\text{盘片数/磁盘})$$

或容量 = 每个扇区可记录的字节数 × 扇区总数，扇区数量由磁道、磁盘面、盘片数、磁盘数决定

磁盘访问



读完蓝色扇区后，逆时针旋转寻找红色扇区的磁道使得读/写头在红色扇区的上方再读红色扇区

访问时间=寻道时间+平均旋转延迟+数据传输时间

寻道时间：读/写头移动到目标柱面所用时间，通常为3-9ms的常数

旋转延迟：旋转盘面使读/写头到达目标扇区上方所用时间。

平均旋转延迟 = $\frac{1}{2} \times \frac{1}{RPMs}$ ，此时的单位是**分钟**，因为RPMs的是转/分钟

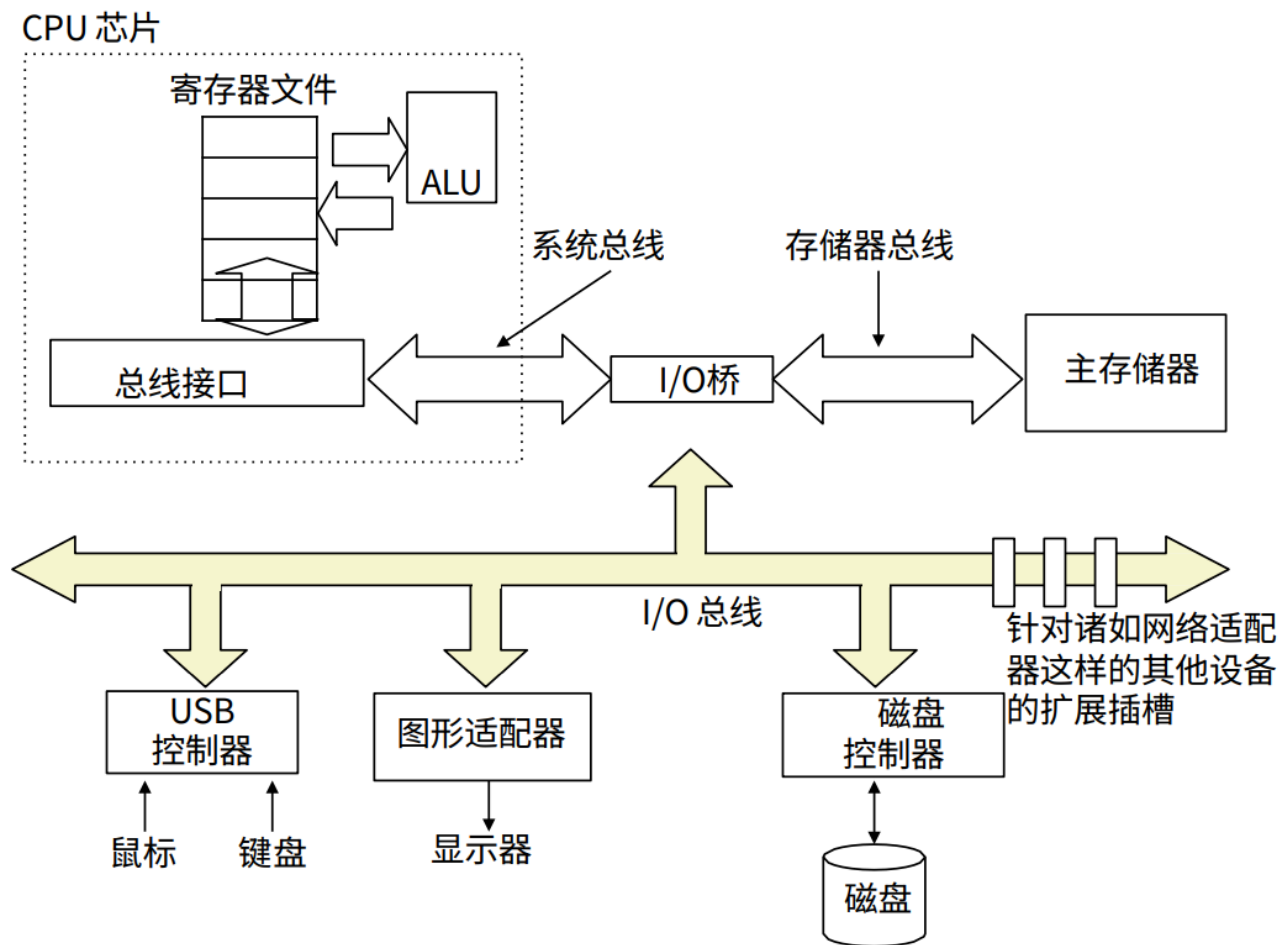
数据传输时间：读目标扇区所用时间

数据传输时间 = $\frac{1}{RPMs} \times \frac{1}{\text{平均扇区数/磁道}}$ ，此时单位也是**分钟**

访问时间主要由**寻道时间**和**旋转延迟时间**组成，相比之下数据传输时间很小

I/O总线

I/O总线 (Input/Output Bus)，是计算机中用于连接 **CPU**和**外部设备**的一种公共通信通道，用于在这些部件之间传输数据、地址和控制信号。



CPU和主存之间的通信是利用**系统总线**和**存储器总线**完成的，而CPU和外设的通信是通过**I/O总线**完成的

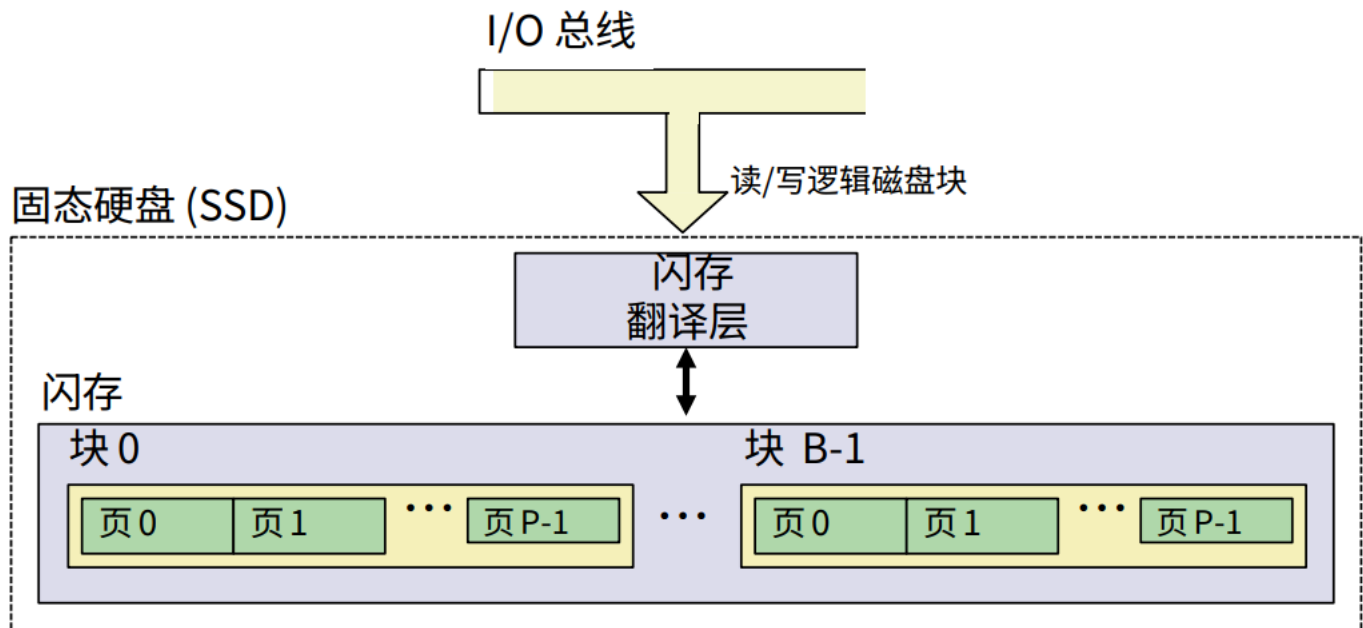
读磁盘扇区的操作

总线接口-磁盘控制器：CPU 通过将命令、逻辑块号和目的内存地址写到与磁盘相关联的内存映射地址，发起一个磁盘读操作

磁盘控制器-主存：磁盘控制器读扇区，并执行到主存的DMA传送（DMA是一种外设直接和主存交换数据的高效数据传输技术，可以释放CPU资源）

磁盘控制器-CPU：当DMA传送完成后，磁盘控制器用中断的方式通知CPU（发送信号到CPU的一个外部引脚）

固态硬盘SSD



闪存：一种非易失性存储器，数据可以反复擦除/写入，速度快于机械硬盘，耐用性高

页（基本存储单元，最小读写单位）大小：512B ~ 4KB，数据以页为单位进行读写

块（最小擦除单位）大小：32 ~ 128页，只有某页所属块整个被擦除后，才能写该页

大约 100,000 次重复写之后，块就会磨损坏

SSD有**顺序访问**和**随机访问**，顺序访问比较快，因为没有寻址开销，也不会写入已使用块中的页和机械硬盘的对比

SSD的优点：没有移动部件，意味着**更快、能耗更低、更结实**

SSD的缺点：磨损更快，更贵

局部性原理

CPU与存储器之间存在较大速度差距，导致CPU频繁等待内存访问，成为系统性能瓶颈。应用**局部性原理**可以缓解这种速度差距带来的问题

- 速度：CPU>DRAM>SSD>磁盘

局部性原理：程序倾向于使用最近一段时间，距离其较近地址的指令和数据。

- **时间局部性：**当前被访问的信息近期很可能还会被再次访问，因此可以将这些频繁访问的数据缓存在**CPU高速缓存**中，减少对较慢主存的访问次数，从而缩短访问延迟，提升CPU利用率。
- **空间局部性：**在最近的将来将用到的信息很可能与现在正在使用的信息在空间地址上是临近的。系统可以**预取相邻数据到缓存**中，提高缓存命中率，降低CPU等待内存访问的时间。
- 例子：

```

sum = 0;
for (i = 0; i < n; i++)
    sum += a[i]
return sum;

```

对数据的引用：步长为1顺序访问（**空间局部性**），变量sum在每次循环迭代中被引用一次（**时间局部性**）

对指令的引用：顺序读取（**空间局部性**），重复执行for循环（**时间局部性**）

- 对于行主序的多维数组，按照行-列的访问顺序会比列-行的访问顺序具有更好的局部性，因为每次访问的是内存中相邻的元素，即访问 `a[i][j]` 后访问的是 `a[i][j+1]`

```

int sum_array_rows(int a[M][N]) {
    int i, j, sum = 0;
    for (i = 0; i < M; i++)
        for (j = 0; j < N; j++)
            sum += a[i][j];
    return sum;
}

int sum_array_cols(int a[M][N]) {
    int i, j, sum = 0;
    for (j = 0; j < N; j++)
        for (i = 0; i < M; i++)
            sum += a[i][j];
    return sum;
}

```

- 让程序局部性尽可能地好的方法：让最内的循环每次数组索引改变的步长尽可能为1。下面这个函数

```

int sum_array_3d(int a[M][N][N]) {
    int i, j, k, sum = 0;
    for (i = 0; i < N; i++)
        for (j = 0; j < N; j++)
            for (k = 0; k < M; k++)
                sum += a[k][i][j];
    return sum;
}

```

访问模式是 `a[k][i][j]`，这会导致较差的局部性，因为最内层循环 `k` 变化时，访问的是不连续的内存位置。为了优化局部性，让最内层循环对应最右边的数组索引。因此，正确的循环顺序应该是 `k、i、j` 变为 `i、j、k`：

```
int sum_array_3d(int a[M][N][N]) {
    int i, j, k, sum = 0;
    for (k = 0; k < M; k++)
        for (i = 0; i < N; i++)
            for (j = 0; j < N; j++)
                sum += a[k][i][j];
    return sum;
}
```

这样就可以使得数组访问模式是步长为1的连续访问，有更好的局部性

高速缓存

Cache：一种更小、速度更快的存储设备。作为更大、更慢存储设备的缓存区。在存储器层次结构中，位于 `k` 层的更快更小存储设备作为位于 `k+1` 层的更大更慢存储设备的缓存。

存储器层次结构的优势：程序 90% 的时间只访问 10% 的数据，通过缓存这 10% 的数据到顶层，大部分访问命中高速存储，整体性能接近顶层速度。同时绝大部分数据存放在底层，使得成本由廉价的大容量存储主导。因此可以实现“像底层存储器一样廉价，又像顶层存储器一样快”

缓存不命中：CPU请求的数据(块A)不在缓存中。此时系统需要将主存中的块A加载到缓存中。

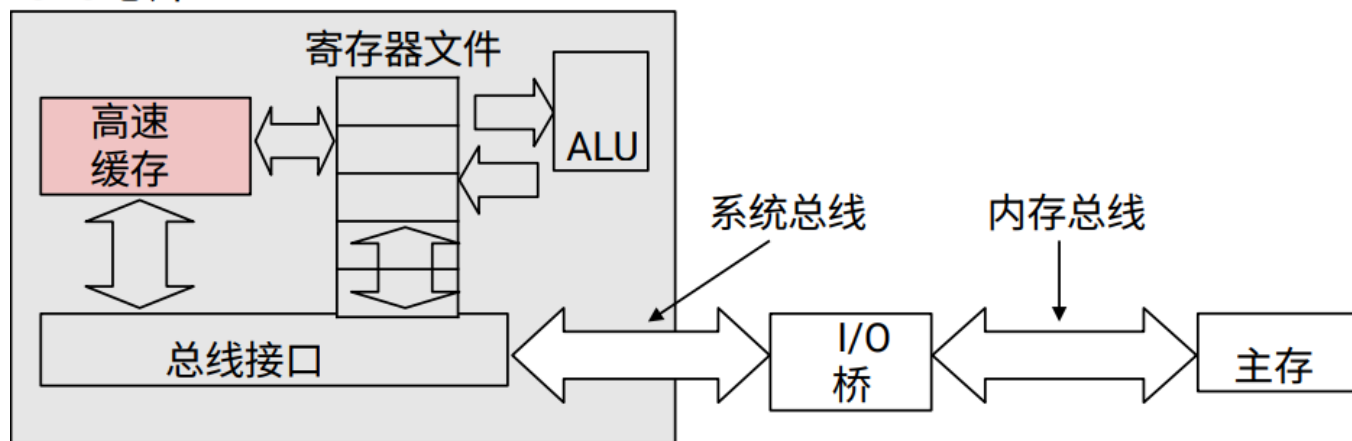
- **放置策略**：决定新加载的块b应该被放置在缓存的哪个位置。常见策略：直接映射（每个内存块只能放缓存特定位置）、全相联（可放任意位置）、组相联（放特定组的任意位置）
- **替换策略**：当缓存已满（或组相联中组已满）时，决定哪个现有缓存块被替换。常见算法：LRU（最近最少使用）、FIFO（先进先出）、随机替换

缓存不命中的种类

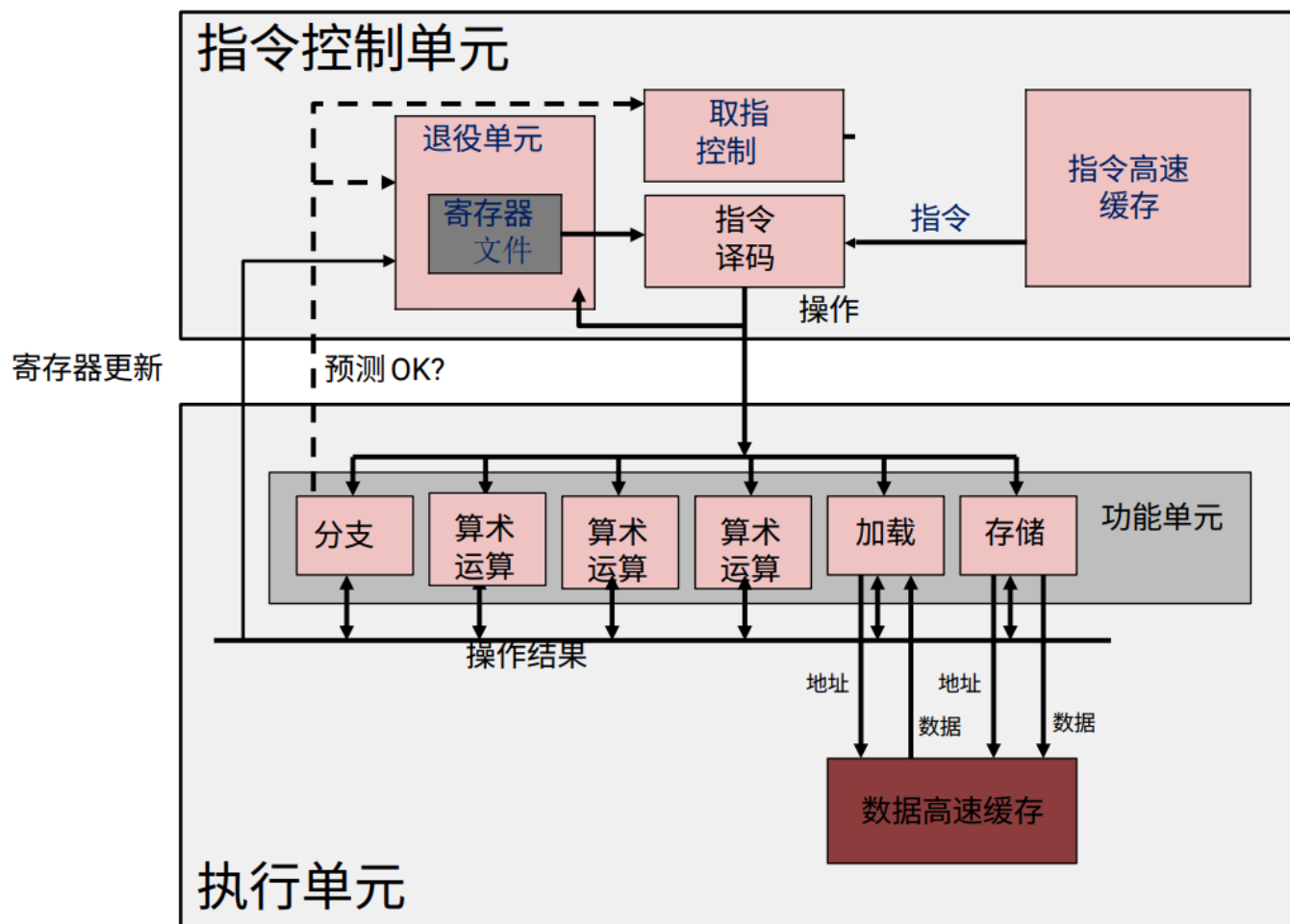
- **冷（强制性）不命中**：当缓存为空时，对任何数据的请求都会不命中
- **冲突不命中**：大多数缓存将第`k+1`层的某个块限制放置在第`k`层块的一个小的子集（或一个块）中。冲突不命中发生在缓存足够大，但是这些多个数据对象会映射到同一个缓存块。比如第`k+1`层的块`i`放在第`k`层的块`(i mod 4)`中，如果程序请求块`0,8,0,8,0,8,...`这样每次引用都会不命中，因为块`0`和块`8`都映射到第`k`层的块`0`
- **容量不命中**：发生在当活跃块集合(working set)的大小比缓存大的时候

高速缓存存储器：小型、快速的基于SRAM的存储器，是在硬件中自动管理的（非用户程序访问的），CPU会首先查找缓存中的数据

CPU 芯片

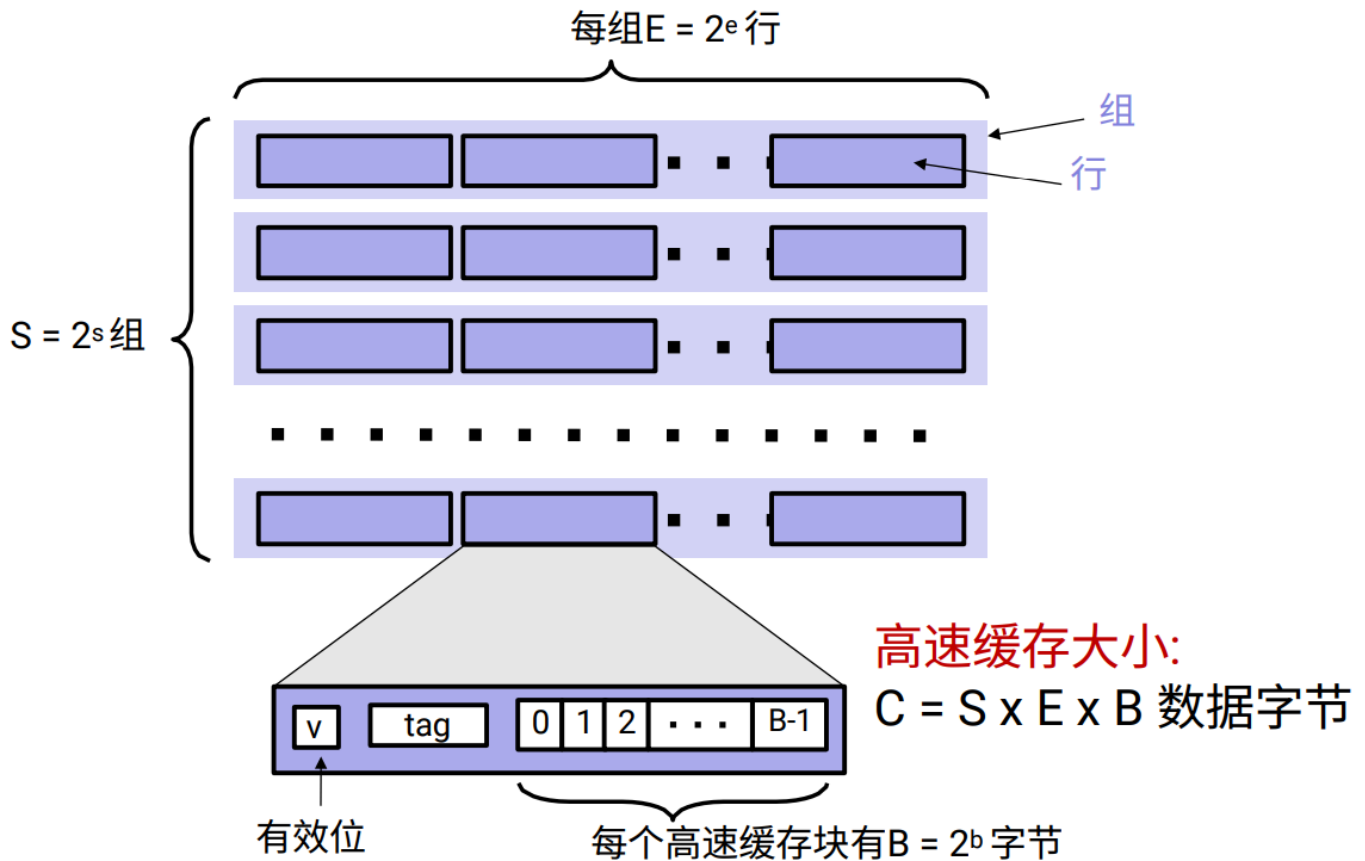


典型的CPU芯片



比较现代的CPU芯片

缓存到主存的三种映射关系：直接映射、组相联映射、全相联映射



算 Tag、SetIndex、Offset 的方法和计组一样， $offset=b, index=s, tag=total-index-offet$
对于直接映射， $E=1$ ；对于全相联映射， $S=1$ ；对于组相联映射，几路组相联E就是几

组相联映射

cache利用率较高
cache冲突率较低(减少了冲突不命中)
淘汰算法较复杂
应用场合：
小容量cache

直接映射

cache利用率很低
cache冲突率很高
淘汰算法很简单
应用场合：
大容量cache

全相联映射(特殊的组相联映射)

cache利用率很高
cache冲突率为零(无冲突不命中)
淘汰算法较复杂
应用场合：小容量cache

看似最好：全相联映射。
但是全相联映射也有缺点：硬件结构很复杂，实现难度和价格都很高，因此实际应用中往往采取组相联映射。

写入策略

- 写命中时

- 直写(优先一致性): 当CPU修改缓存中的数据时, 同时立即将数据写入主存
- 写回(优先性能): 当CPU修改缓存中的数据时, 仅更新缓存, 并标记该缓存块为"脏" (Dirty Bit)。只有当该缓存块被替换时, 才将数据写回主存。
- 写不命中时
 - 写分配 (搭配写回): 写失效时先加载数据到缓存再修改, 更多的写遵循局部性
 - 非写分配 (搭配直写): 写失效时直接写主存, 不加载缓存

不命中率的计算

1. 确定缓存块的大小
2. 确定访问模式, 包括访问的步长和访问的数据总量
3. 计算一个缓存块能容纳的数据元素数量
4. 判断哪些访问会不命中
5. 计算不命中率

如果缓存块大小比为A(通常>8)字节, 向量V元素大小为B字节, 对v的步长为C。

每个块的元素数量 = $\lfloor \frac{A}{B} \rfloor$, 通常 $A \bmod B = 0$, 当作 $\frac{A}{B}$

访问第一个元素 $v[0]$ 时, 缓存不命中, 加载整个缓存块 (A字节); 后续访问的地址如果仍在缓存块内, 则命中。每个块总共有 $\frac{A}{B}$ 个元素, 步长为C, 可以命中 $\frac{\frac{A}{B}}{C}$ 次。缓存块用完后又不命中, 加载新的缓存块, 重复这个过程。

因此每 $\frac{A}{B \times C}$ 次访问不命中一次, 不命中率为 $\frac{B \times C}{A}$ 。

特别的, 如果 $C \geq A$, 每次都不命中, 不命中率为1

若高速缓存的块大小为 64 字节, 向量 v 的元素为 int, 则对v 的步长为1 的应用的不命中率为
每个块元素=64/4=16个, 在 $v[0]$ 时不命中, 加载缓存块可以命中到 $v[15]$, 共16次不命中1次。从 $v[16]$ 又重复这个过程, 所以不命中率是1/16

重新排列以提升空间局部性

以矩阵乘法为例子, 假设块大小为32字节

原始顺序(ijk/jik)

```
for (i=0; i<n; i++) {
    for (j=0; j<n; j++) {
        sum = 0.0;
        for (k=0; k<n; k++)
            sum += a[i][k] * b[k][j];
        c[i][j] = sum;
    }
}
```

分析最内层循环。对于行访问的a数组, 缓存块可存4个 double, 不命中率为0.25。对于列访问的b数组, 每次都要跨行加载新的缓存块, 不命中率为1。对于c数组, 仅在k循环结束后被写入一

次，不命中率为0。因此此方法每次内循环迭代需要**2次加载**（`a[i][k]` and `b[k][j]`），**0次存储**（`c[i][j]` 是在k循环外面写的）。不命中率/每次迭代=1.25

优化循环顺序(`kij/ikj`)

```
for (k=0; k<n; k++) {
    for (i=0; i<n; i++) {
        r = a[i][k];
        for (j=0; j<n; j++)
            c[i][j] += r * b[k][j];
    }
}
```

固定 `k` 和 `i`，内层循环 `j` 连续访问 `b[k][j]`和`c[i][j]`，空间局部性好。每次内循环迭代需要**2次加载**（`a[i][k]` and `b[k][j]`），**1次存储**（`c[i][j]`）。`a`、`b`、`c` 的不命中率分别为0、0.25、0.25，总共不命中率/每次迭代=0.5

较差的循环顺序(`jki/kji`)

```
for (j=0; j<n; j++) {
    for (k=0; k<n; k++) {
        r = b[k][j];
        for (i=0; i<n; i++)
            c[i][j] += a[i][k] * r;
    }
}
```

内层循环 `i` 访问 `a[i][k]` 和 `c[i][j]` 的列，均为非连续内存访问（跨步访问），空间局部性差。每次内循环迭代需要**2次加载**（`a[i][k]` and `b[k][j]`），**1次存储**（`c[i][j]`）。`a`、`b`、`c` 的不命中率分别为1、0、1，总共不命中率/每次迭代=2

使用块来提高时间局部性

还是以矩阵乘法为例子，这次假设缓存块为64字节，缓存大小 $C \ll n$

```
c = (double *) calloc(sizeof(double), n*n);

/* Multiply n x n matrices a and b */
void mmm(double *a, double *b, double *c, int n) {
    int i, j, k;
    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++)
```



```

        for (k = 0; k < n; k++)
            c[i*n + j] += a[i*n + k] * b[k*n + j];
    }

```

访问模式：a[i*n + k] 按行访问，具有良好的空间局部性。b[k*n + j] 是按列访问的，每次访问 b[k][j] 都会跳转n个元素（8n字节），导致缓存不命中。c[i*n + j] 是在最内层循环中固定不变的，只有在外层i和j变化时才改变，每次写入 c[i][j] 时可能已经不在缓存中。不命中的原因除了b的按列访问外，还有当n非常大时，a的行和b的列可能无法同时保存在缓存中，导致反复的缓存替换。

不命中分析：对于第一次迭代，a的不命中次数为n/8（每个块8个double，步长为1），b的不命中次数为n（列访问），c的不命中次数只有 c[0][0] 一次，忽略。不命中次数 $\frac{n}{8}(a) + n(b) = \frac{9n}{8}$ 。后续迭代内层替换k的访问方式和第一次一样，由于缓存大小 $C \ll n$ ，每次外层 i 或 j 变化之前加载的 a 或 b 的数据已被挤出缓存，无法复用。总不命中次数 $n \times n \times \frac{9n}{8} = \frac{9}{8}n^3$ 。

分块后，每个块 B×B，一共三个块，为了同时留在缓存中，需要有 $3B^2 < C$

```

c = (double *) calloc(sizeof(double), n*n);

/* Multiply n x n matrices a and b */
void mmm(double *a, double *b, double *c, int n) {
    int i, j, k;
    for (i = 0; i < n; i+=B)
        for (j = 0; j < n; j+=B)
            for (k = 0; k < n; k+=B)
                /* B x B mini matrix multiplications */
                for (i1 = i; i1 < i+B; i1++)
                    for (j1 = j; j1 < j+B; j1++)
                        for (k1 = k; k1 < k+B; k1++)
                            c[i1*n+j1] += a[i1*n + k1]*b[k1*n + j1];
}

```

访问模式：外循环：以B为步长遍历 i, j, k，将问题分解为多个B×B的子矩阵乘法。内循环：对于每个块，进行类似于直接乘法的计算，但范围限制在B×B的块内。优势在于，虽然b仍然是跨行访问，但因为块的大小B通常使得B×B的子矩阵可以放入缓存，整个b的块可以保留在缓存中。因此，对于固定的k块和j块，b[k1][j1] 的访问可以在缓存中找到。同时c也可以留在缓存中。

不命中分析：每块不命中次数 $\frac{B^2}{8}$ ，每次迭代要加载a和b两个块，外层循环 n/B 次，每次外层循环不命中次数 $2 \times \frac{n}{B} \times \frac{B^2}{8} = \frac{nB}{4}$ 次。外层循环总次数 (n/B)×(n/B)，总不命中次数 $\frac{nB}{4} \times (\frac{n}{B})^2 = \frac{n^3}{4B}$ 次

B 越大，每个块的计算量越大，数据复用率更高，因此应该选择 满足 $3B^2 < C$ 的最大可能 B

策略总结

- 集中在内部循环上，大部分计算和内存访问都发生在这里。
- 尽量按照数据对象存储在内存中的顺序，以步长为1来读数据，使得程序的空间局部性最大。
- 一旦从内存中读入一个数据对象，尽可能多使用它，使得程序中的时间局部性最大